eBPF 在链路追踪中应用与实践

匠心
杭州乘云数字技术有限公司

陕西 · 西安　2026年4月18日

01

链路追踪：分布式系统的“透视眼”

DISTRIBUTED SYSTEM TRACING

链路追踪的核心价值

服务依赖可视化

自动生成的服务调用拓扑图，让我们对整个系统的架构和依赖关系一目了然，有助于架构评审和优化。

容量规划与优化

通过分析历史调用数据和性能指标，可以预测系统的瓶颈所在，为资源扩容和优化提供数据支撑。

快速故障定位

当系统出现问题时，通过查看完整的调用链，可以迅速定位到错误发生的具体服务和环节，大大缩短排查时间。

性能瓶颈分析

精确测量每个Span的耗时，帮助我们找到系统中最耗时的操作，从而进行针对性的优化。

好代码是调试出来的，

好产品是观测出来的。

典型应用场景

微服务架构下的性能调优与故障排查

应对复杂微服务环境，帮助开发和运维人员快速定位性能瓶颈与故障根源。

电商大促等高峰期的系统稳定性保障

流量洪峰下实时监控系统状态，快速发现异常链路，保障核心业务稳定运行。

复杂分布式事务的监控与分析

完整记录跨服务事务的执行过程与耗时，确保数据一致性与事务完整性。

多团队协作开发中的责任边界划分

通过清晰的调用链路映射，明确各团队负责的服务范围，提升跨团队协作效率。

第三方服务调用的性能与可靠性评估

量化评估外部依赖服务的响应时间与可用性，为技术选型和服务优化提供依据。

应用性能 / 链路追踪 / 调用链详情

最近1小时

错误 GET /methodA6 | demo-service-a::k8s

开始时间：2024-06-25 12:30:00　总调用耗时：5.90ms　TraceID：8856640351205779591

性能分解

External 2,381.138 ms
Code 14.557 ms
Database 3.465 ms
网络 2.153 ms
NoSQL 2.017 ms
Pool 0.022 ms

性能分解统计

乘云数字

调用树

调用次序　火焰图　耗时统计　服务流

请搜索名称...

	执行占比	0ms 2ms 4ms 6ms 8ms 10ms
GET /methodA6 demo-service-a::k8s | Web | web	16.72%	9.20ms
GET /methodA6 demo-service-a::k8s | Web | web	13.31%	8.30ms
org.springframework.web.util.Nested demo-service-a::k8s | Web | web	8.23%	6.10ms
java.lang.RuntimeException demo-service-a::k8s | Web | web	6.20%	5.10ms
lo.grpc.StatusRuntimeExceptio demo-service-a::k8s | Web | we	2.40%	1.34ms 0.01ms 0.43ms
org.springframework.web.client.HttpSer demo-service-a::k8s | Web | web	8.23%	2.60ms
lo.grpc.StatusRuntimeException demo-service-a::k8s | Web | web	6.20%	6.20ms
org.springframework.web.util.Nested demo-service-a::k8s | Web | web	8.40%	8.40ms
java.lang.RuntimeException demo-service-a::k8s | Web | web	5.20%	5.20ms
lo.grpc.StatusRuntimeExceptio demo-service-a::k8s | Web | we	4.10%	4.10ms
org.springframework.web.client.HttpSer demo-service-a::k8s | Web | web	5.20%	5.20ms
lo.grpc.StatusRuntimeException demo-service-a::k8s | Web | web	4.10%	4.10ms
java.lang.RuntimeException demo-service-a::k8s | Web | web	3.20%	3.20ms
lo.grpc.StatusRuntimeException	4.10%	

WEB　数据库　缓存　MQ　自定义　错误Span

demo-service:k8s　kafka.deliv > ka...

标签　基础设施　指标　日志

系统信息

主机名	k8s-a3
操作系统	GNU/Linux
系统内核	5.8.13-1.el7.elrepo.x86_64
系统架构	x86_64
系统版本	CentOS Linux-7 (Core)

内存

内存大小	31.37 GiB
交换字节总数	0 B
其它信息	其它信息001

02

主流实现技术：从侵入式到无侵入式

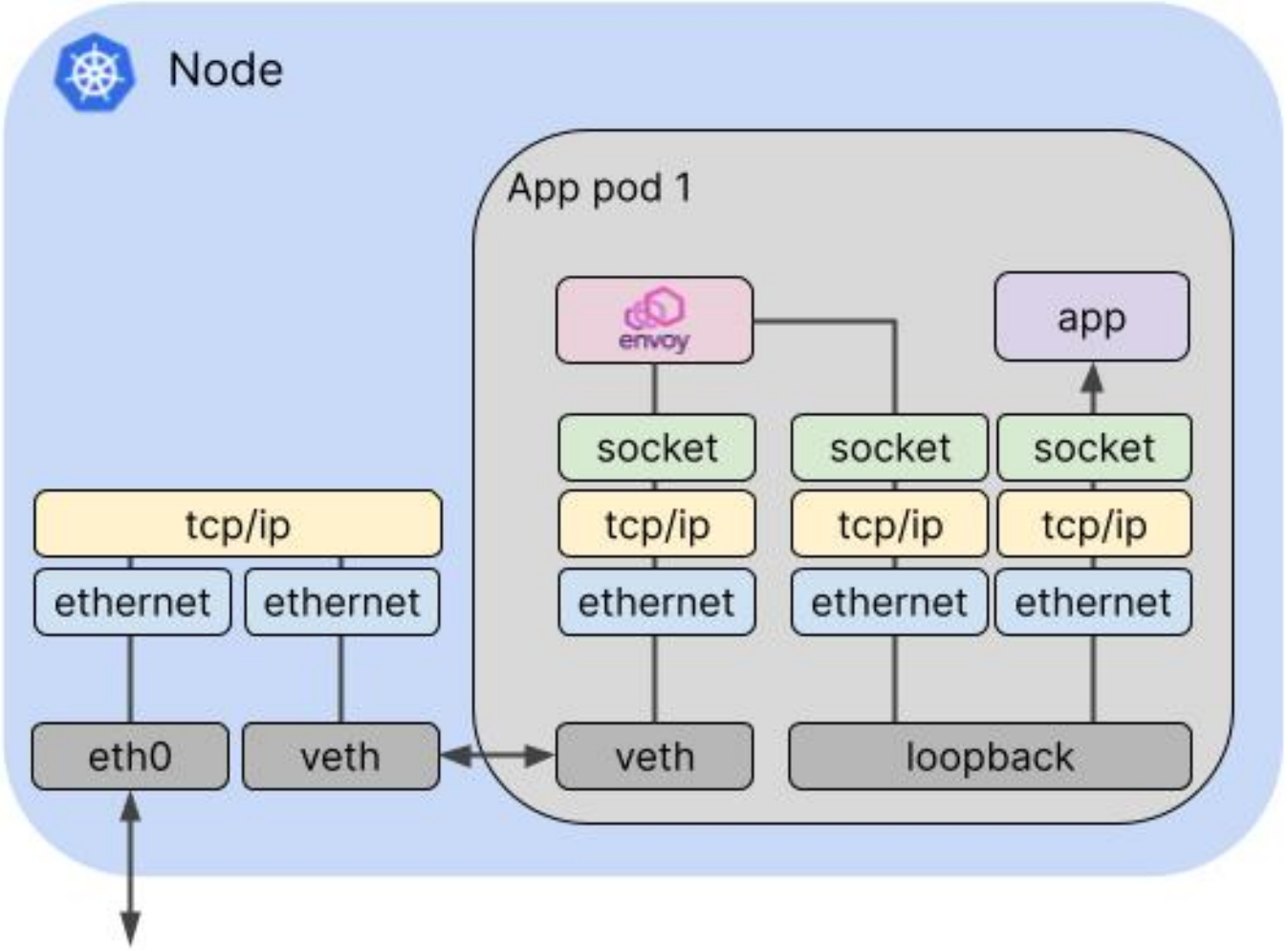
MAINSTREAM IMPLEMENTATION TECHNOLOGIES

主流实现技术对比

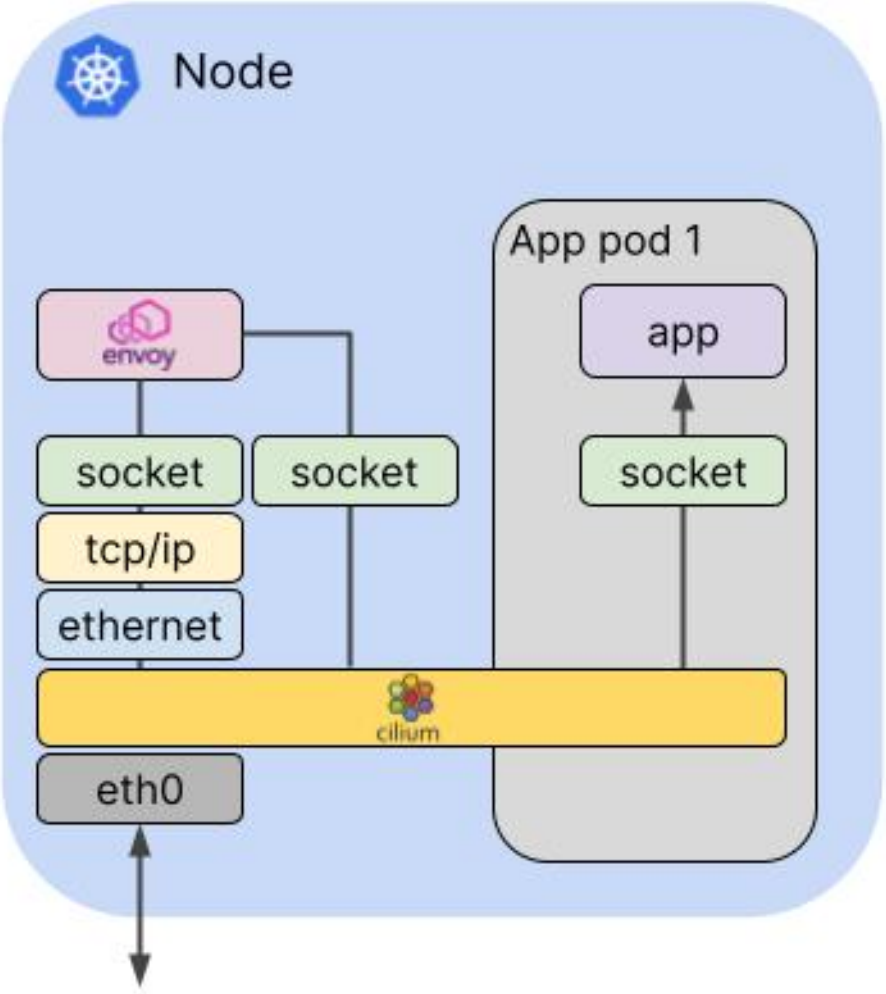
技术类型	实现方式	代表工具	优点	缺点
代码埋点	手动或半自动在代码中插入追踪逻辑	各种JavaAgent	灵活、精准，可自定义追踪数据	侵入性强，对不同编程语言、框架的兼容性，开发和维护成本高
边车代理	通过独立的代理服务拦截流量并注入追踪信息	Istio Linkerd	对应用无侵入，统一管理	增加部署复杂度和资源开销，可能成为瓶颈
旁路流量	通过旁路方式抓取流量，分析流量数据	NPM	对应用无侵入，不需要重启应用，对环境要求极低	上下文信息极少，流量解析开销大，短连接丢失
内核态追踪(eBPF)	通过内核技术在底层捕获调用信息，无需修改应用	DeepFow Pixie	无侵入，上下文感知丰富，不需要重启应用。	技术门槛较高，对内核版本有要求

链路追踪技术演进趋势：从侵入式到无侵入式，从应用层到内核层

Service mesh with traditional networking



Sidecarless model, eBPF acceleration



03

eBPF 实现链路追踪技术解析

EBPF TECHNICAL ANALYSIS: THE "SUPER PROBE" OF KERNEL SPACE

基于eBPF的K8s链路追踪工作流程

1. 部署eBPF Agent

以DaemonSet方式在K8s集群每个Node上部署Agent。

2. 内核Hook点注入

加载eBPF程序至内核，挂载到网络(TC/XDP)及系统调用钩子。

3. 数据采集

捕获系统调用中的数据、网络包与系统事件

4. 数据处理（协议分析）

从HTTP/gRPC头部自动提取或生成Trace ID、Span ID。

5. 数据聚合、关联与上报

关联Pod/Service等K8s元数据。
NAT 转换

6. 可视化展示

通过UI工具查看完整调用链路、性能指标及服务拓扑。

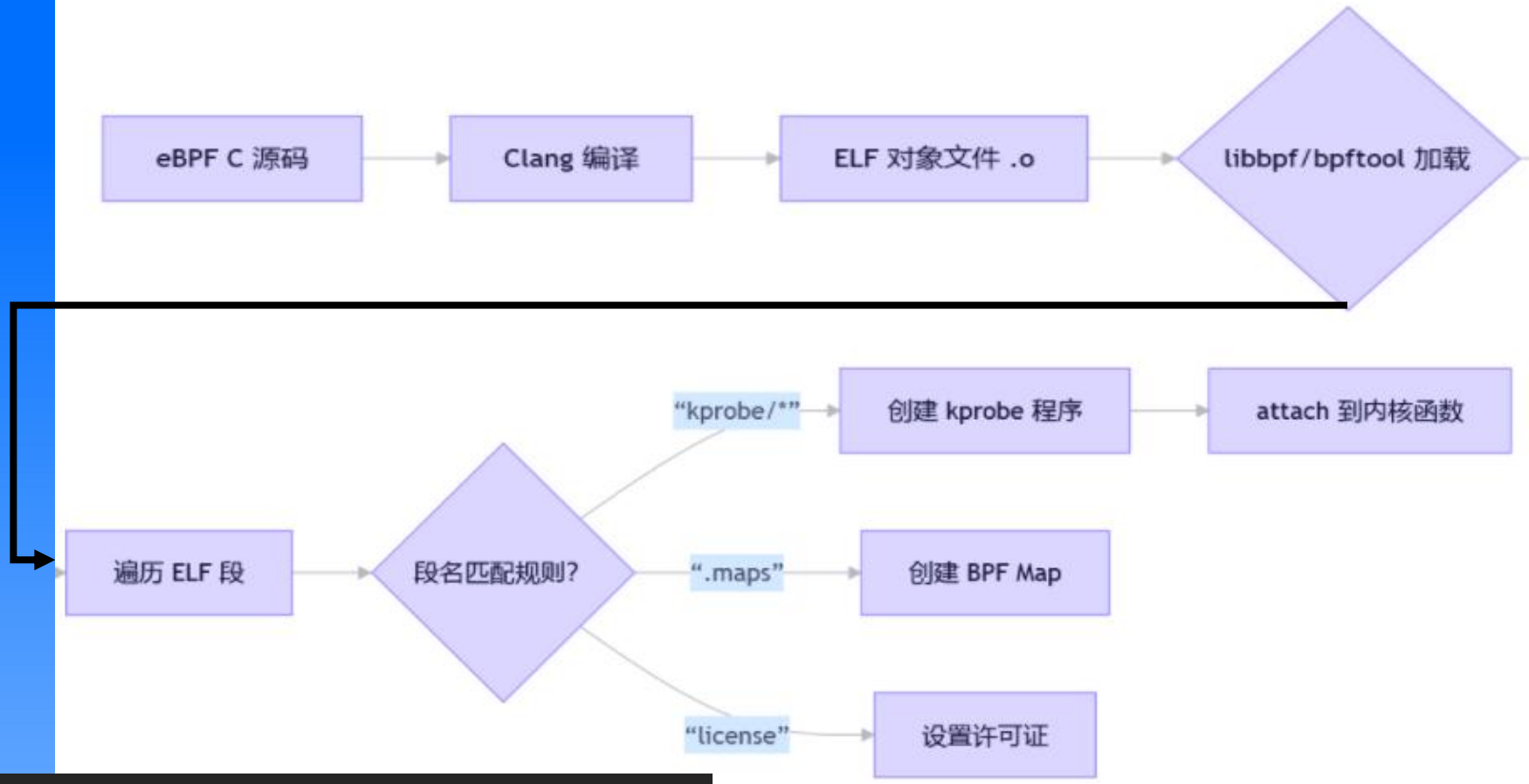
04

挑战与解决

CHALLENGES AND SOLUTIONS: TRACING DIFFICULTIES UNDER NAT

影响bpf校验的因素

- Clang版本
- 编译选项
- Libbpf版本
- 内核



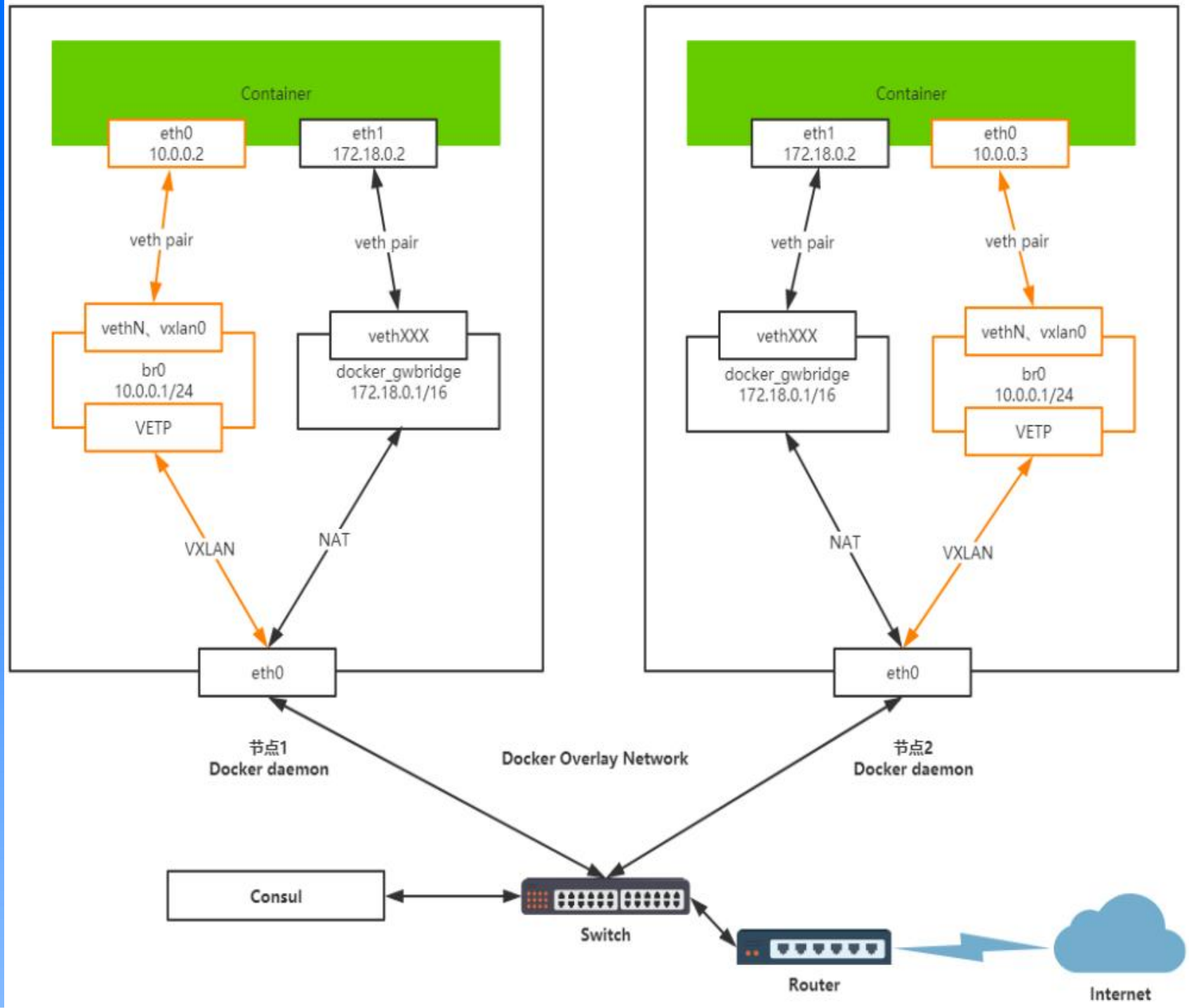
错误信息	原因	解决方案
`R1 invalid mem access 'inv'`	未初始化的寄存器访问	初始化所有寄存器，特别是函数调用后
`same insn cannot be used with different pointers`	指针类型混用	不要在不同分支中重用指令处理不同指针类型
`invalid access to stack R1 off=-8 size=8`	栈越界访问	确保栈分配足够，访问在边界内
`Unreleased reference`	资源泄漏	函数退出前释放所有引用（socket/map）
`infinite loop detected at insn X`	无限循环	确保循环计数器正确有界并修改
`R1 type=inv expected=ctx`	类型不匹配	确保寄存器类型匹配helper期望
`invalid read from stack R1 off=-8`	未初始化栈读取	初始化栈内存后再读取

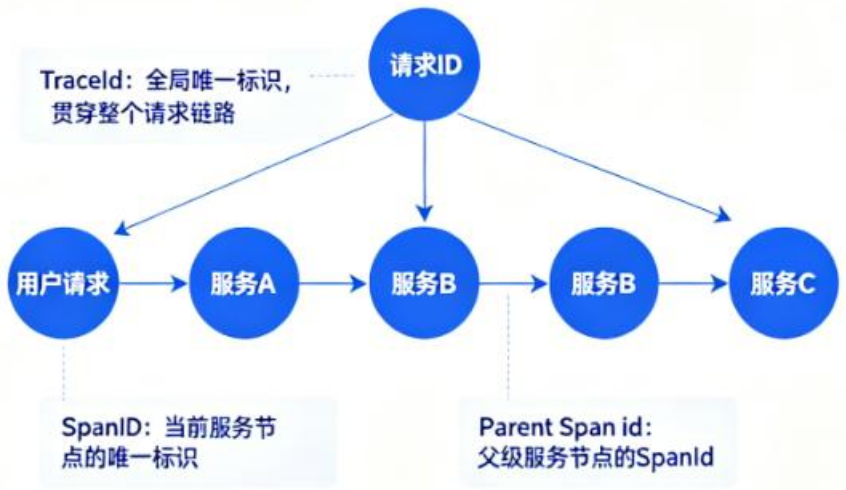
不同场景对NAT的处理

- 虚拟机
- Docker
- K8S
- 中间设备

不同组件对NAT的处理

- Netfilter/nf_conntrack
- Netfilter/ipvs
- docker-proxy
- nginx
- cilium





- **TraceId**

  唯一标识整个请求链路，所有节点共享同一TraceId

- **SpanId**

  每个服务节点生成唯一SpanId，标识当前节点

- **Parent Span id**

  指向父级服务节点的SpanId，形成调用层级关系

技术方案	ID的存储	ID的传递	ID的生成
JavaAgent	线程局部存储	进程内上下文/协议头	随机生成
eBPF	BPF_HASH_MAP	无	计算生成

https://community.databuff.com

感谢聆听

Q & A

欢迎提问与交流